

JAVA CORE PROGRAMMING

Comprehensive Technical Reference & Study Guide

PART 1: FUNDAMENTALS

1. JAVA ARCHITECTURE & EXECUTION MODEL

Java is a high-level, class-based, object-oriented programming language designed to have as few implementation dependencies as possible. Key to its portability is the "Write Once, Run Anywhere" (WORA) philosophy enabled by the Java Virtual Machine (JVM).

JDK vs JRE vs JVM

JVM (Java Virtual Machine): Executes bytecodes. Contains JIT compiler & Garbage Collector.

JRE (Java Runtime Environment): JVM + Core Class Libraries required to execute Java apps.

JDK (Java Development Kit): JRE + Development tools (javac, debugger, javadoc).

Compilation Workflow

- Source code (.java) is compiled by javac into Bytecode (.class).
- JVM loads bytecode via ClassLoader.
- Bytecode is interpreted and translated to native machine code by JIT Compiler.

2. PRIMITIVE DATA TYPES & MEMORY USAGE

Type	Size	Default Value	Range / Value Set
byte	8-bit	0	-128 to 127
short	16-bit	0	-32,768 to 32,767
int	32-bit	0	-2^{31} to $2^{31} - 1$
long	64-bit	0L	-2^{63} to $2^{63} - 1$
float	32-bit	0.0f	IEEE 754 floating point
double	64-bit	0.0d	IEEE 754 floating point
char	16-bit	'\u0000'	0 to 65,535 (Unicode)
boolean	1-bit (logical)	false	true or false

3. THE 4 PILLARS OF OBJECT-ORIENTED PROGRAMMING

Encapsulation

DATA HIDING

Abstraction

HIDING COMPLEXITY

Inheritance

CODE REUSABILITY

Polymorphism

MANY FORMS

```
// Basic Class Syntax Demonstrating Encapsulation
public class BankAccount {
    private double balance; // Encapsulated state

    public BankAccount(double initialBalance) {
        if (initialBalance >= 0) this.balance = initialBalance;
    }

    public void deposit(double amount) {
        if (amount > 0) balance += amount;
    }
}
```

JAVA CORE PROGRAMMING

OOP Mechanics & Memory Management

PART 2: OOP & MEMORY

1. POLYMORPHISM & METHOD OVERLOADING VS OVERRIDING

Compile-time (Overloading)

- Same method name, different parameter signature.
- Occurs within the same class.
- Return type can be same or different.
- Resolved during compilation.

Runtime (Overriding)

- Same method name and exact parameters.
- Occurs across subclass and superclass.
- Return type must be covariant.
- Resolved dynamically at runtime via vtable.

```
class Animal {
    void makeSound() { System.out.println("Generic animal sound"); }
}

class Dog extends Animal {
    @Override
    void makeSound() { System.out.println("Bark bark"); }
    void makeSound(int count) { /* Overloaded */ }
}
```

2. ABSTRACT CLASSES VS INTERFACES

Feature	Abstract Class	Interface (Java 8+)
Inheritance	Single inheritance (extends)	Multiple inheritance (implements)
State / Fields	Can have instance fields & constructors	Only public static final constants
Methods	Abstract and concrete methods	Abstract, default, static, private
Speed	Slightly faster method dispatch	Interface lookup overhead

3. JVM MEMORY ARCHITECTURE: STACK VS HEAP

Stack Memory

- Stores local variables and primitive types.
- Stores references to objects in Heap.
- Thread-safe; thread has its own call stack.
- LIFO structure; automatic allocation/deallocation.

Heap Memory

- Stores all created objects and arrays.
- Shared across all threads.
- Managed by the Garbage Collector (GC).
- Subdivided: Young Gen (Eden, S0, S1) & Old Gen.

Garbage Collection Overview

The Garbage Collector automatically reclaims memory occupied by unreachable objects. Common GC algorithms include G1GC (Garbage-First), ZGC, and Parallel GC. Explicit `System.gc()` calls only request GC; execution is never guaranteed.

JAVA CORE PROGRAMMING

Exception Handling, Generics & I/O

PART 3: EXCEPTIONS & I/O

1. THROWABLE CLASS HIERARCHY

All exceptions and errors in Java descend from `java.lang.Throwable`.

Checked Exceptions

Subclasses of `Exception` (excluding `RuntimeException`). Must be caught or declared in `throws` block.

Examples: `IOException`, `SQLException`, `ClassNotFoundException`.

Unchecked Exceptions & Errors

Subclasses of `RuntimeException` and `Error`. Indicate bugs or unrecoverable conditions.

Examples: `NullPointerException`, `OutOfMemoryError`.

2. TRY-WITH-RESOURCES (AUTOCLOSEABLE)

Introduced in Java 7, `try-with-resources` ensures that every resource implementing `java.lang.AutoCloseable` is automatically closed at execution end.

```
// Automatic resource closure prevents memory/file descriptor leaks
public void readFile(String path) {
    try (BufferedReader br = new BufferedReader(new FileReader(path))) {
        String line;
        while ((line = br.readLine()) != null) {
            System.out.println(line);
        }
    } catch (IOException e) {
        System.err.println("Error reading file: " + e.getMessage());
    }
}
```

3. JAVA GENERICS & WILDCARDS

Generics provide type safety and eliminate the need for manual casting at runtime.

- `Box<T>` — Generic Type parameter.
- `List<?>` — Unbounded Wildcard (Read-only access).
- `List<? extends Number>` — Upper Bounded Wildcard (Covariance: Read items as `Number`).
- `List<? super Integer>` — Lower Bounded Wildcard (Contravariance: Safe to add `Integers`).

PECS Principle: Producer Extends, Consumer Super

Use `? extends T` when you only read from a data structure (Producer). Use `? super T` when you only write to a data structure (Consumer).

JAVA CORE PROGRAMMING

Java Collections Framework (JCF)

PART 4: COLLECTIONS

1. COLLECTIONS HIERARCHY OVERVIEW

The Java Collections Framework provides architecture to store and manipulate groups of objects. Root interfaces are Collection and Map.

2. COLLECTION TYPES & BIG-O COMPLEXITY

Data Structure	Ordering	Duplicates?	Get / Access	Insert / Delete
ArrayList	Insertion Order	Yes	O(1)	O(N)
LinkedList	Insertion Order	Yes	O(N)	O(1)
HashSet	None	No	O(1)	O(1)
TreeSet	Sorted / Natural	No	O(log N)	O(log N)
HashMap	None	Keys No, Val Yes	O(1)	O(1)
TreeMap	Sorted Keys	Keys No, Val Yes	O(log N)	O(log N)

3. HASHMAP INTERNAL MECHANICS

HashMap operates on the principle of Hashing. It stores data in key-value pairs inside an array of Node<K,V> buckets.

Collision Resolution

When two keys generate the same bucket index (hash collision), Java uses a Linked List to chain entries.

Treeification Threshold

Since Java 8, if bucket list size exceeds **8** and array capacity ≥ 64 , list converts into a Red-Black Tree (improving performance from O(N) to O(log N)).

```
// Demonstrating Core Map Operations
Map<String, Integer> map = new HashMap<>();
map.put("Alice", 85);
map.put("Bob", 92);

// Compute if absent pattern
map.computeIfAbsent("Charlie", k -> 78);

for (Map.Entry<String, Integer> entry : map.entrySet()) {
    System.out.println(entry.getKey() + " => " + entry.getValue());
}
```

JAVA CORE PROGRAMMING

Multithreading & Concurrency

PART 5: CONCURRENCY

1. THREAD STATES & LIFECYCLE

A Java thread undergoes state transitions during execution managed by the Thread Scheduler.

- **NEW:** Thread instance created, `start()` not yet called.
- **RUNNABLE:** Executing in the JVM or waiting for OS resources.
- **BLOCKED:** Waiting for a monitor lock to enter a synchronized block.
- **WAITING:** Waiting indefinitely for another thread to perform an action (e.g., `Object.wait()`).
- **TIMED_WAITING:** Waiting for a specified time (e.g., `Thread.sleep(ms)`).
- **TERMINATED:** Run method has completed execution.

2. SYNCHRONIZATION & LOCK MECHANICS

Synchronized Keyword

Acquires intrinsic monitor lock. Ensures mutual exclusion and visibility of changes across memory (Happens-Before relationship).

Volatile Keyword

Guarantees variable visibility by reading/writing directly to Main Memory, bypassing CPU cache. Does **not** ensure atomicity.

3. EXECUTOR FRAMEWORK & THREAD POOLS

Manages worker threads efficiently without the creation overhead per task.

```
import java.util.concurrent.*;

public class ConcurrencyExample {
    public static void main(String[] args) throws Exception {
        ExecutorService executor = Executors.newFixedThreadPool(2);

        Future<String> future = executor.submit(() -> {
            Thread.sleep(500);
            return "Task Completed";
        });

        System.out.println(future.get()); // Blocks until result is ready
        executor.shutdown();
    }
}
```

Atomic Classes

Classes like `AtomicInteger`, `AtomicReference` use hardware-level Compare-And-Swap (CAS) instructions to achieve non-blocking concurrency without lock synchronization.

JAVA CORE PROGRAMMING

Modern Java Features & Code Quality

PART 6: MODERN FEATURES

1. FUNCTIONAL PROGRAMMING & LAMBDAS (JAVA 8+)

Functional interfaces (containing exactly one abstract method) enable functional programming expressions.

- `Predicate<T>` — Takes `T`, returns boolean (e.g., `t -> t.isEmpty()`).
- `Function<T, R>` — Takes `T`, returns `R` (e.g., `t -> t.length()`).
- `Consumer<T>` — Takes `T`, returns void (e.g., `t -> System.out.println(t)`).
- `Supplier<T>` — Takes no args, returns `T` (e.g., `() -> Math.random()`).

2. STREAM API OPERATIONS

```
List<String> names = Arrays.asList("Alice", "Bob", "Charlie", "Anna");

List<String> filtered = names.stream()
    .filter(n -> n.startsWith("A"))
    .map(String::toUpperCase)
    .sorted()
    .collect(Collectors.toList());
// Output: ["ALICE", "ANNA"]
```

3. JAVA 14+ RECORDS & PATTERN MATCHING

Immutable data carrier classes and simplified pattern matching syntax.

```
// Compact immutable data class
public record Point(int x, int y) {}

// Pattern Matching for instanceof (Java 16+)
public void process(Object obj) {
    if (obj instanceof String s) {
        System.out.println("Length: " + s.length());
    }
}
```

4. ESSENTIAL JAVA BEST PRACTICES SUMMARY

Do's

- Use `StringBuilder` for string manipulation in loops.
- Favor immutability (final fields, Records).
- Override `equals()` whenever overriding `hashCode()`.
- Use `Optional<T>` to represent missing return values safely.

Don'ts

- Don't catch generic `Throwable` or swallow exceptions.
- Avoid raw types; use proper Generic parameters.
- Don't use `float` or `double` for precise financial calculations (Use `BigDecimal`).

